Demeure AI: An Intelligent Campus Operating System with Hybrid AWS Cloud Integration

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology
School of Electronics Engineering (SENSE)

by

Pratik Kumar (24BML0027)
Thakur Chand Choudhary (24BML0012)
Nikunj Goydani (24BEC0338)

Under the guidance of

Dr. Rajalaxmi S

Assistant Professor Sr. Grade 1
School of Electronics Engineering (SENSE)
Vellore Institute of Technology, Vellore – 632 014



October, 2026

Executive Summary

Hostel and campus administration in many Indian universities still relies on paper outpass registers, complaints posted in chat groups, roommate surveys kept in spreadsheets and manually reconciled dues. Approvals are slow, safety-critical issues go unnoticed and there is no audit trail. This project presents Demeure AI, a role-based campus operating system for administrators, wardens, mess managers and students. A React 19 single-page application communicates with a FastAPI service that verifies every sign-in on the server, issues signed HttpOnly session cookies and enforces role-based access on every endpoint. A hybrid storage layer runs durably on SQLite and switches to Amazon DynamoDB, S3, SES and Polly when AWS credentials are configured. Intelligent features include a retrieval-augmented handbook assistant, weighted roommate matching over a 100-student survey dataset, rule-based complaint triage and a SHA-256 hash-chained gate ledger. In evaluation, the assistant retrieved the correct rulebook for 18 of 18 questions and answered 17 correctly; triage accuracy rose from 93.3% to 100% on a 30-complaint sample after a matching fix; core API calls completed within 3 ms at the 95th percentile; and 32 automated tests and ten end-to-end scenarios passed. The work supports SDGs 3, 4, 7, 12 and 16.

CONTENTS

Page No.

Executive Summary . . . . . **ii**

1. INTRODUCTION **1**

1.1 Literature Review 1

1.2 Research Gap 2

1.3 Problem Statement 3

1.3.1 Relevance of the Problem Statement w.r.t. SDG 3

2. PROJECT OBJECTIVE **4**

3. PROPOSED WORK **5**

3.1 Design Approach / System Model / Algorithm 5

3.2 Technical Descriptions 9

4. HARDWARE/SOFTWARE TOOLS USED **16**

5. RESULT ANALYSIS **17**

5.1 Functional verification 17

5.2 Automated tests 17

5.3 API performance 18

5.4 Handbook assistant 19

5.5 Roommate compatibility on the 100-student dataset 19

5.6 Complaint triage 20

5.7 Engineering quality 20

5.8 Discussion 21

6. CONCLUSION AND FUTURE WORK **22**

6.1 Summary 22

6.2 Limitations and Constraints 22

6.3 Improvement / Future Work 23

7. SOCIAL AND ENVIRONMENTAL IMPACT **24**

8. WORK PLAN **25**

8.1 Timeline 25

8.2 Individual Contribution 25

9. COST ANALYSIS **26**

10. REFERENCES **27**

11. APPENDIX-1: SOURCE CODE **29**

1. INTRODUCTION

Large residential universities in India house thousands of students in hostels that operate like small towns. Wardens approve leave and visitor requests, maintenance teams respond to electrical and plumbing faults, mess managers plan menus for hundreds of diners, finance offices compute and collect hostel, electricity and mess dues, and grievance cells handle sensitive complaints such as ragging. In most institutions these processes are spread across paper registers, messaging groups, spreadsheets and e-mail. Information is duplicated, approvals depend on who happens to be available, and urgent issues such as a sparking socket can sit unread in a chat thread.

Demeure AI is a web-based campus operating system that brings these processes onto one platform. It gives four personas (Campus Administrator, Hostel Administrator/Warden, Mess Manager and Resident Student) a workspace scoped to their responsibilities and keeps every record in one persisted source of truth. The platform began as a prototype for the WebForge Smart Campus Webathon 2026 and was then engineered for production: server-side authentication, role-based authorisation, durable storage, input validation, automated tests and a deployable container image. It also integrates four Amazon Web Services (DynamoDB, S3, SES and Polly) through a hybrid design that runs entirely on one machine when cloud credentials are absent.

This report describes the problem, the related literature, the system design and its algorithms, the tools used, and a quantitative evaluation of the implemented system, followed by limitations, impact, the work plan and a cost analysis.

1.1 Literature Review

Smart campuses. Min-Allah and Alrashed describe the smart campus as an integration of sensing, networking and data-driven services that improves resource use, safety and the student experience [20]. Zanella et al. show how Internet-of-Things infrastructures support city-scale services such as energy monitoring and building management [21]. These architectures emphasise data collection; comparatively little attention is given to the administrative workflows (approvals, grievances, billing) that consume most of a hostel office's day.

Retrieval-augmented assistants. Large language models answer fluently but can invent facts. Lewis et al. introduced retrieval-augmented generation (RAG), which conditions generation on passages retrieved from a trusted corpus [1]. Dense retrieval relies on sentence embeddings such as Sentence-BERT [2]; MiniLM distils transformer models [4] into compact encoders that run on a CPU [3]. Open-weight models such as gpt-oss-20b [25] make it practical to ground answers in an institution's own rulebooks at low cost.

Matching and allocation. Roommate assignment is a classic problem: Gale and Shapley formalised stable matching [18], and Irving gave an efficient algorithm for the stable roommates problem [19]. Hostel offices, however, rarely run formal matching; they need a transparent ranking that a warden can inspect and override. A weighted similarity over a small set of lifestyle attributes meets that need and is easy to explain.

Anomaly detection and energy. Isolation Forest [5] is widely used to flag unusual readings in utility telemetry without labelled data. In this project, power telemetry is simulated, so the dashboard demonstrates the user workflow (inspect, resolve, bill) rather than a trained detector.

Tamper evidence. Haber and Stornetta showed that chaining document hashes makes backdating detectable [6]; the same idea underpins Bitcoin's block chain [7]. With SHA-256 [8], a simple hash chain gives a gate log tamper evidence without a distributed ledger.

Secure web applications. OWASP lists broken access control and cryptographic failures as the top web risks [13]. NIST SP 800-63B [12] and RFC 8018 (PBKDF2) [10] guide password storage, RFC 2104 defines HMAC message authentication [9], RFC 6265 governs cookies [11] and Content Security Policy limits injected scripts [14].

Cloud data services. Amazon's Dynamo paper established the highly available key-value model behind DynamoDB [15], and the AWS Well-Architected Framework sets out the security, reliability and cost pillars used to evaluate cloud deployments [17]. REST principles [16] shape the platform's resource-oriented API.

Policy context. The UGC anti-ragging regulations require institutions to provide confidential reporting channels [22]. The UN 2030 Agenda frames campus efficiency, safety and inclusion as part of the Sustainable Development Goals [23], and UNEP reports that food waste is a significant share of food production [24], which is relevant to institutional mess halls.

1.2 Research Gap

The literature and existing hostel tools leave the following gaps, which this project targets:

- **Workflow over sensing.** Smart-campus work concentrates on sensors and dashboards, while the approval, grievance and billing workflows that need cross-role coordination are rarely addressed end to end.
- **Access control as an afterthought.** Many student-built portals enforce roles only in the user interface, so any user who calls the API directly can read or change other users' data.
- **Ungrounded assistants.** Generic chatbots answer policy questions without citing the institution's own rules.
- **Opaque allocation.** Roommate allocation is either manual or treated as a black box, with no explainable score or enforcement of institutional constraints such as single-gender

blocks.

- **Cloud lock-in.** Cloud-native designs often cannot run without paid services, which blocks evaluation and low-budget deployments; this project shows the same code running locally or on AWS.

1.3 Problem Statement

Design and implement a secure, role-based web platform that digitises the core hostel and campus workflows (leave and visitor approvals, room allocation, complaints, mess planning, grievances, facilities, billing and payment), keeps a single persisted record shared across roles, provides an AI assistant grounded in institutional rulebooks, and can be deployed either on a single low-cost server or on AWS managed services without code changes.

1.3.1 Relevance of the Problem Statement w.r.t. SDG

Table 1.1 maps the platform's features to the Sustainable Development Goals and the specific targets they address.

Table 1.1: Mapping of Demeure AI features to the Sustainable Development Goals

SDG	Target addressed	Demeure AI contribution
SDG 3: Good Health and Well-being	3.4 Promote mental health and well-being	Welfare early-warning alerts, anonymous grievance channel, emergency complaint escalation
SDG 4: Quality Education	4.a Safe, non-violent, inclusive learning environments	Anti-ragging reporting, visitor pre-authorisation, accountable leave approvals
SDG 7: Affordable and Clean Energy	7.3 Improve energy efficiency	Per-zone power monitoring, anomaly resolution and usage-based billing
SDG 12: Responsible Consumption	12.3 Halve food waste	Published weekly menus, meal feedback loop and wastage fines
SDG 16: Strong Institutions	16.6 Accountable, transparent institutions	Role-based approvals, tamper-evident gate ledger, auditable records

2. PROJECT OBJECTIVE

The objectives of the project are:

1. To build a single web platform that serves four personas (Campus Administrator, Hostel Warden, Mess Manager and Student) with role-scoped modules, covering 13 operational modules and an overview dashboard.
2. To enforce security on the server: hashed credentials, signed expiring sessions, brute-force throttling and role and ownership checks on every API endpoint.
3. To persist every record in a store that runs on SQLite locally and on Amazon DynamoDB in the cloud, with private file storage on local disk or Amazon S3.
4. To provide a retrieval-augmented handbook assistant that answers from the institution's rulebooks and cites its sources, with speech output through Amazon Polly or the browser.
5. To rank compatible roommates from student survey responses (seeded with a 100-student dataset) using an explainable weighted score that respects single-gender blocks.
6. To triage maintenance complaints automatically into category, priority and response-time targets.
7. To make security events tamper-evident with a SHA-256 hash-chained gate ledger.
8. To verify the system with automated tests and scripted end-to-end scenarios, and measure its latency and accuracy.
9. To package the platform for production with a container image, health checks and continuous integration.

3. PROPOSED WORK

The proposed system is a three-tier web application: a React single-page client, a FastAPI application service, and a data and cloud tier with interchangeable local and AWS adapters. This chapter describes the system model and algorithms (Section 3.1) and the implemented modules with screenshots (Section 3.2).

3.1 Design Approach / System Model / Algorithm

3.1.1 System architecture

Figure 3.1 shows the architecture. The browser loads a code-split React application; each of the 13 modules is downloaded only when its tab is opened. All requests go to the same origin, so the session cookie is sent automatically and cross-site requests are blocked by the SameSite=Strict attribute. The FastAPI service is organised into six routers (auth, hostel, campus, billing, aws and chat). Each router depends on a shared service container that holds the authentication service, the document store, the AWS adapter and the retrieval-augmented assistant.

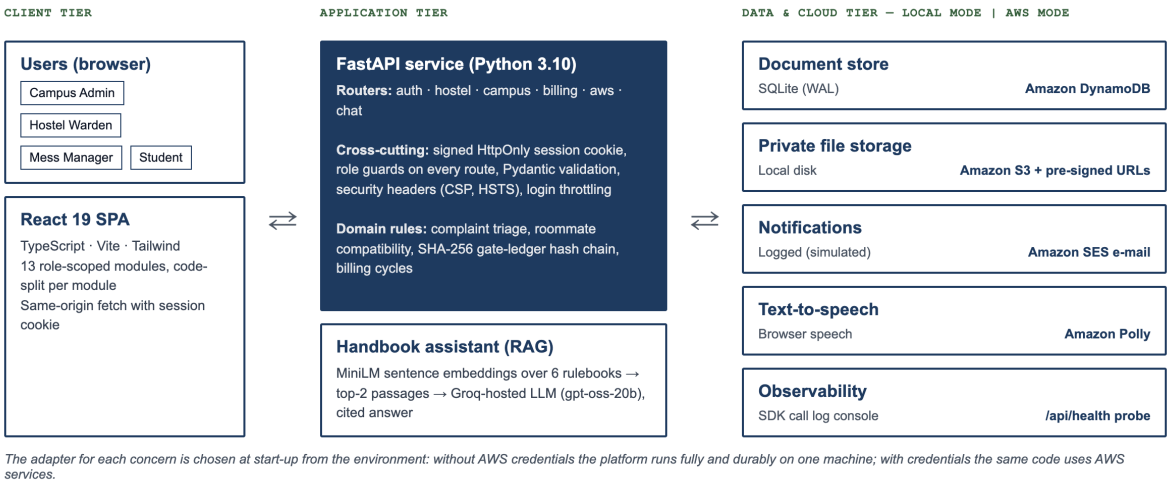


Fig. 3.1: *Three-tier architecture of Demeure AI with local and AWS adapters*

The data and cloud tier is selected at start-up. When AWS credentials are present the store uses DynamoDB (one table per collection, created on demand with on-demand billing), uploads go to a private S3 bucket and are served through five-minute pre-signed URLs, notifications are sent through SES and speech is synthesised by Polly. Without credentials the same interfaces use SQLite in write-ahead-logging mode, the local disk, a logged e-mail simulation and the browser's speech synthesis. Every call is written to an observable log that administrators can watch in the AWS Console Hub.

3.1.2 Role model and access control

Authorisation is enforced on the server for every route; the user interface only hides what the server would refuse anyway. Table 3.1 summarises which persona can use each module. Students can see only their own leaves, visitors, surveys, complaints and grievances. Staff responses never reveal who filed an anonymous grievance.

Table 3.1: Role-based access to modules

Module	Campus Admin	Hostel Warden	Mess Manager	Student
Overview telemetry and AI handbook assistant	Yes	Yes	Yes	Yes
Academic hub, facilities and fleet, finance and campus life	Yes	–	Finance only	Yes
Power grid diagnostics and billing dispatch	Yes	–	–	–
AWS Console Hub	Yes	–	–	–
Room allocation, survey responses, security gate, welfare	–	Yes	–	Survey form only
NLP complaint desk	Yes (manage)	Yes (manage)	–	Yes (own tickets)
Mess and food optimiser	–	–	Yes (publish)	Yes (view, rate)
Student services: dues, payment, leave, visitor pass	–	–	–	Yes

3.1.3 Data model

All records are JSON documents with a string identifier, stored in named collections (Table 3.2). The same document shape is written to SQLite or DynamoDB; floating-point values are converted to DynamoDB decimals transparently. Server-generated identifiers such as `L-3F9A2C` use a cryptographically random suffix, so IDs cannot collide under concurrent requests.

3.1.4 Algorithms

(a) Credential verification and sessions. Staff PINs and student access codes are stored only as PBKDF2-HMAC-SHA256 hashes with a random 16-byte salt and 120,000 iterations [10], and are compared in constant time. After a successful login the server issues a token

$$t = \mathrm{B64}(p) \parallel \text{“.”} \parallel \mathrm{B64}\big(\mathrm{HMAC}_{\mathrm{SHA256}}(K,\ \mathrm{B64}(p))\big), \tag{3.1}$$

where the payload p holds the user identity, role and an expiry time and K is the server’s secret key [9]. The token is set as an HttpOnly, SameSite=Strict cookie [11]. On each request the signature is recomputed and compared in constant time; a modified payload (for example a

Table 3.2: Principal collections

Collection	Key fields	Written by
leaves	studentId, startDate, endDate, reason, proofUrl, status	Student (create), Warden (approve/deny)
visitors	hostId, visitor, date, purpose, status, passCode	Student (request), Warden (approve/deny)
billing / payments	powerRate, baseFee, cycleId / receiptId, amount, cycleId	Admin (dispatch) / Student (pay)
surveys	ownerId, lifestyle answers, preferred block and floor	Student (upsert); 100 seeded students
room_allocations	occupants, maxOccupants	Warden
complaints, maintenance, bookings, grievances	text, category, priority, status, filedBy	All roles (create), staff (status)
mess_menu, mess_reviews	weekly days / rating, meal, suggestion	Mess Manager / Student
gate_ledger	seq, time, subject, status, prevHash, hash	Warden (gate events)
files	key, ownerId, contentType	Upload endpoint

forged role) or an expired token is rejected with HTTP 401. Ten failed logins from one client within five minutes return HTTP 429.

(b) Retrieval-augmented handbook assistant. Each rulebook is split into sections at level-two headings, giving 30 chunks across six rulebooks, and each chunk is embedded with the all-MiniLM-L6-v2 encoder [2], [3]. For a query q the assistant computes the cosine similarity against every chunk c,

$$\mathrm{sim}(q, c) = \frac{\mathbf{e}_q \cdot \mathbf{e}_c}{\|\mathbf{e}_q\| \, \|\mathbf{e}_c\|}, \tag{3.2}$$

keeps the top two with similarity above 0.2, and passes them to the language model with the instruction to answer only from that context [1]. The answer is returned with the cited file names. If no model is configured, the retrieved policy text is returned verbatim, so the feature degrades gracefully.

(c) Roommate compatibility. Each lifestyle dimension d (sleep, study, cleanliness, social) is mapped to an ordinal scale $\{0, 1, 2\}$. For the warden's requested profile a and a resident's survey b, the score is

$$S(a, b) = 100 \times \frac{\sum_d w_d \, (1 - |a_d - b_d|/2)}{\sum_d w_d}, \tag{3.3}$$

with weights $w = (0.30, 0.25, 0.25, 0.20)$. Residents who prefer the same block receive a +5 bonus, capped at 100. Blocks A and B are single-gender, so when the target block is A or B only residents who chose that block are eligible; Block C, which is mixed, ranks all residents. The top three candidates are returned with their scores.

(d) Complaint triage. Each category (Electrical, Plumbing, Structural, IT/WiFi) has a keyword list. A keyword counts as a hit only when it occurs at the start of a word, so "lock" does not match "Block". The category with the most hits wins; ties go to the more safety-critical category, and with no hits the complaint is General/Low. If any hazard term (socket, spark, shock, fire, smoke, gas, flood and similar) appears, the priority is raised to CRITICAL with a two-hour response target.

(e) Tamper-evident gate ledger. Gate events form a hash chain. With $h_0 = 0^{64}$ and canonical JSON $C(e)$ of the fields time, subject, type, purpose and status, each entry stores

$$h_i = \text{SHA-256}\big(h_{i-1} \,\|\, C(e_i)\big) \tag{3.4}$$

[6], [8]. Verification recomputes the chain on every read; editing, reordering or deleting any past entry changes all later hashes, and the warden console shows an integrity failure.

(f) Billing cycles and idempotent payment. Every rate dispatch by the administrator creates a new cycle identifier. The student's statement is computed on the server as base fee + metered kWh × rate + mess fines, so the client cannot change the amount. A payment is stored under the key (student, cycle), which makes a second payment in the same cycle impossible (HTTP 409) while a new cycle reopens the dues.

3.1.5 Cross-role workflow

Figure 3.2 traces the leave-approval workflow across the student, the API, the storage and notification services and the warden. The other workflows (visitor passes, complaints, grievances, bookings) follow the same pattern: create by one role, a validated state transition by another, and a notification.

#	Student (browser)	Demeure API	Store / S3 / SES	Warden (browser)
1	**Selects dates, reason and a PDF proof**			
2	POST /api/aws/s3/upload	Checks session, type, magic bytes, 5 MB limit; stores privately	Object saved; file owner recorded	
3	POST /api/leaves/submit	Validates dates; name and room taken from session; attachment ownership checked	Leave saved (status Pending); guardian e-mailed via SES	
4		GET /api/leaves (HostelAdmin role) returns all leaves		**Sees pending queue**
5		GET /api/files/{key}: owner or staff only (pre-signed URL on AWS)	Proof streamed / redirected	**Opens the proof**
6		POST /api/leaves/{id}/approve: only from Pending (409 otherwise)	Status Approved; student e-mailed via SES	**Approves**
7	**"My Applications" shows Approved**	GET /api/leaves returns only this student's leaves		

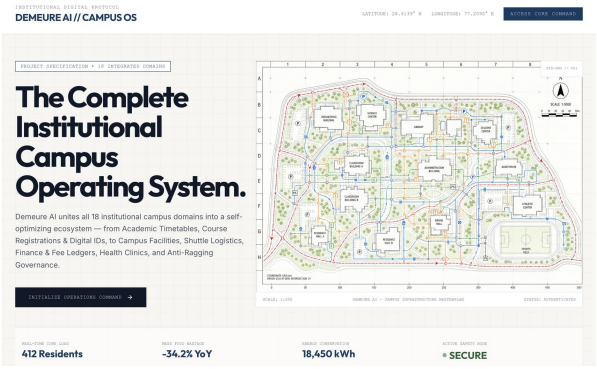
Fig. 3.2: *Leave request with a private proof attachment, from submission to approval*

3.2 Technical Descriptions

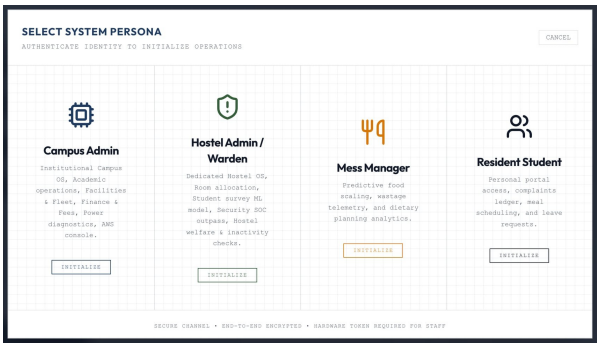
This section walks through the implemented modules using screenshots captured from the running application by an automated Chrome session (Puppeteer) signed in as each persona. The complete source code is in the project repository; key algorithms are reproduced in Appendix-1.

3.2.1 Landing page and authentication

The landing page (Fig. 3.3a) introduces the platform and leads to the persona picker (Fig. 3.3b). Staff authenticate with a 12-digit PIN on an on-screen keypad and students with an institutional e-mail and access code. Wrong credentials are rejected by the server. In production builds the demo hints are hidden, and the server refuses to start with demo PINs or a weak signing key.



(a) *Landing page*



(b) *Persona selection*

Fig. 3.3: *Entry to the platform*

3.2.2 Student services: dues, payment, leave and visitor passes

The student portal (Fig. 3.4) shows a server-computed statement for the current billing cycle and accepts a UPI payment, after which a receipt is stored against the cycle (Fig. 3.5b). Leave applications validate dates and accept a PDF, PNG or JPG proof of up to 5 MB, verified by its file signature and stored privately (Fig. 3.5a). Students can also request visitor passes; on approval they receive a gate pass code (Fig. 3.7).

3.2.3 Hostel security operations

The warden's security console lists pending leaves with a link to the private attachment (Fig. 3.6a) and pending visitor requests. Approve and deny are one-way transitions; acting twice returns HTTP 409. The biometric gate simulator writes entries to the hash-chained ledger, whose verification status is shown in the console (Fig. 3.6b).

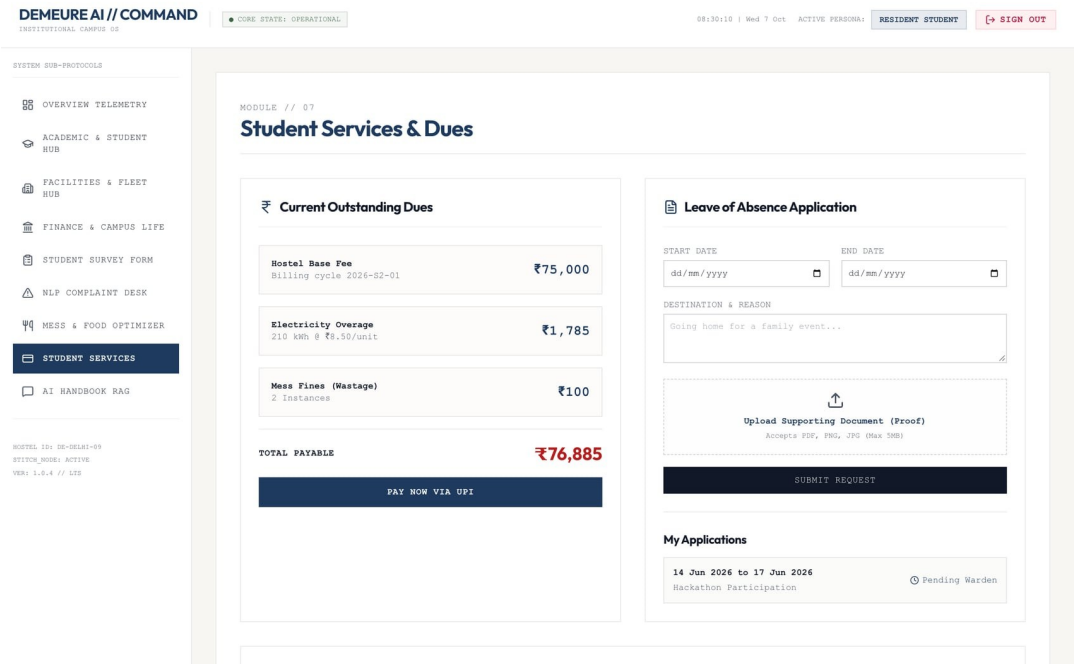
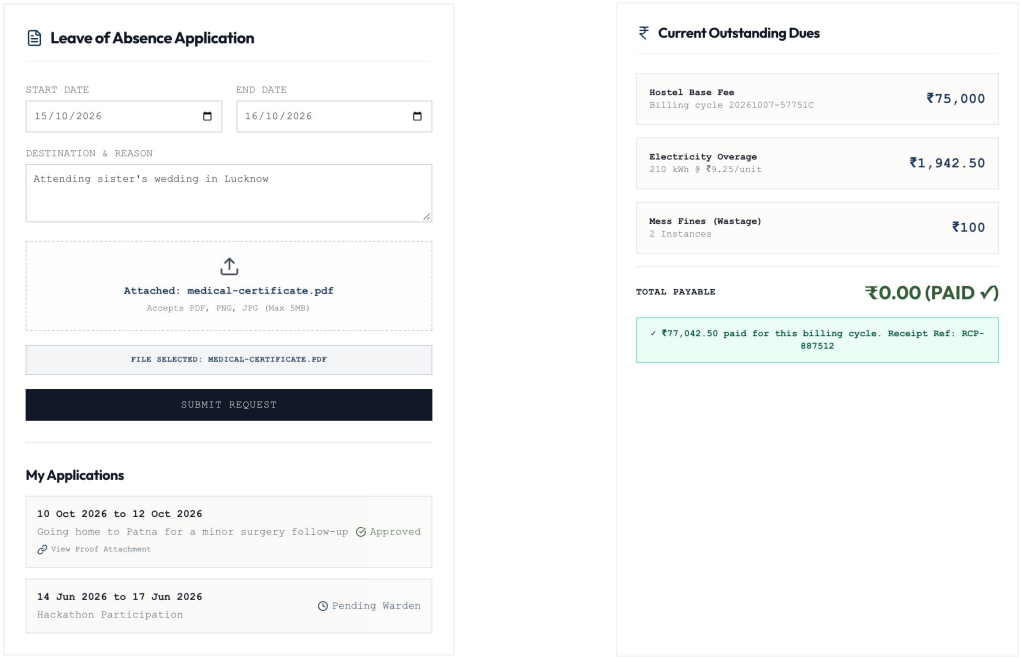
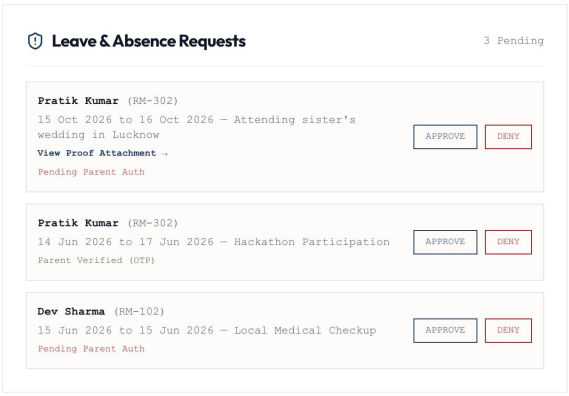


Fig. 3.4: *Student services portal*

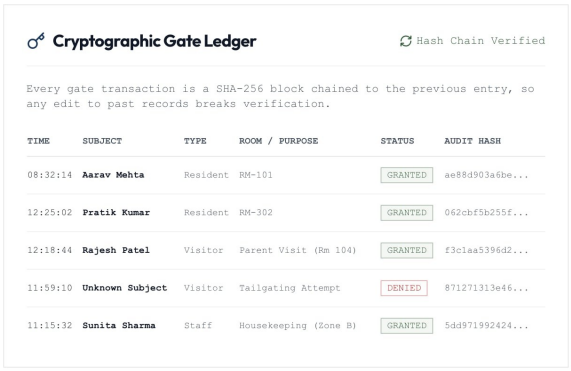


(a) *Leave application with proof attached* (b) *Dues settled with a stored receipt*

Fig. 3.5: *Student self-service*



(a) *Warden leave queue with proof attachment*



(b) *Hash-chained gate ledger (chain verified)*

Fig. 3.6: *Warden security console*

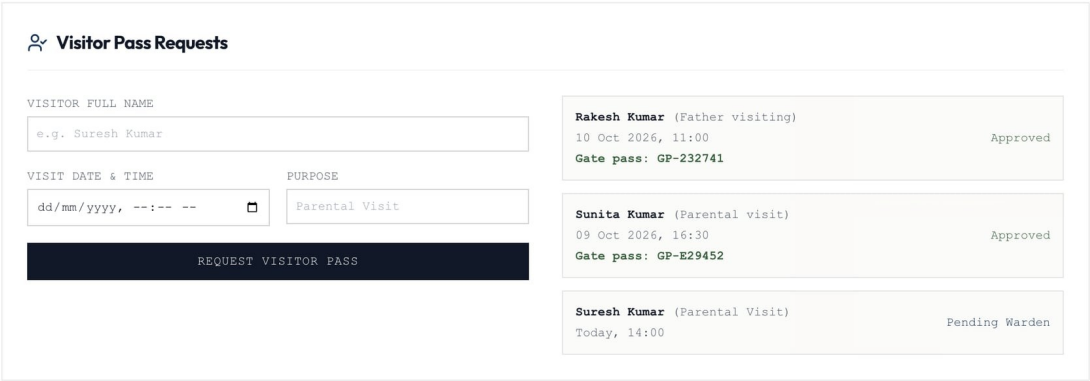


Fig. 3.7: *Visitor pass requests with issued gate pass codes*

3.2.4 Survey and room allocation

Students submit a room-preference survey that is saved on the server. Wardens see every response; the 100 seeded responses plus new submissions feed the compatibility engine (Fig. 3.8). Allocations are checked for capacity and duplicates on the server and persisted, so every warden session sees the same occupancy.

3.2.5 Decision-support engines

The compatibility scorer ranks candidate roommates by the weighted score of Section 3.1.4(c) (Fig. 3.9a). On the complaint desk, a student describes a problem in free text and the triage engine predicts its category, priority and response target before the ticket is logged (Fig. 3.9b); staff then move tickets from Open to Dispatched to Resolved. The handbook assistant answers policy questions with citations to the source rulebook (Fig. 3.10), and a speaker button reads the answer aloud through Amazon Polly or the browser's speech synthesis.

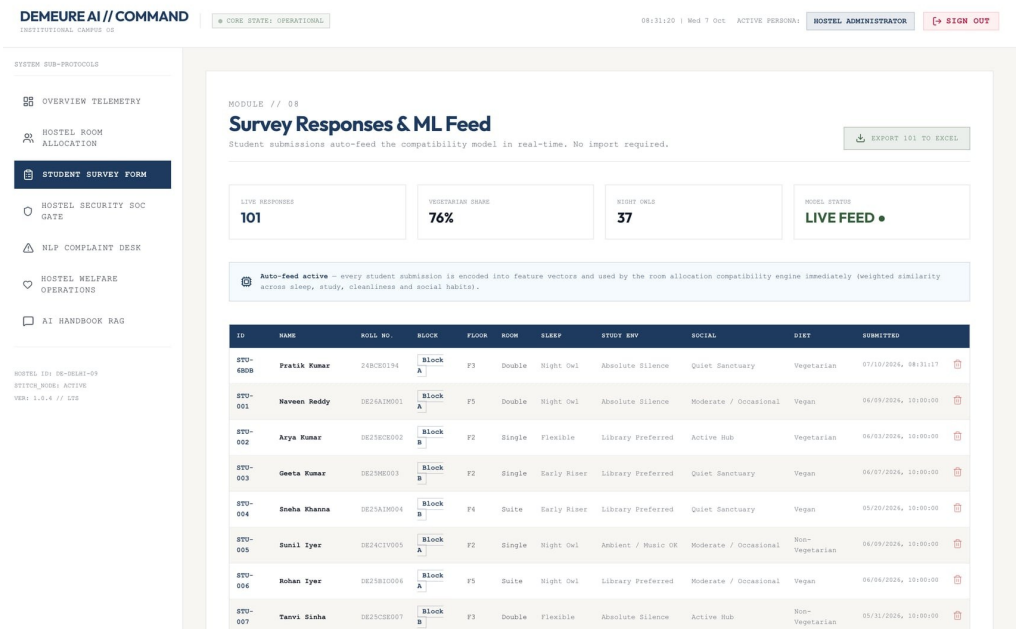
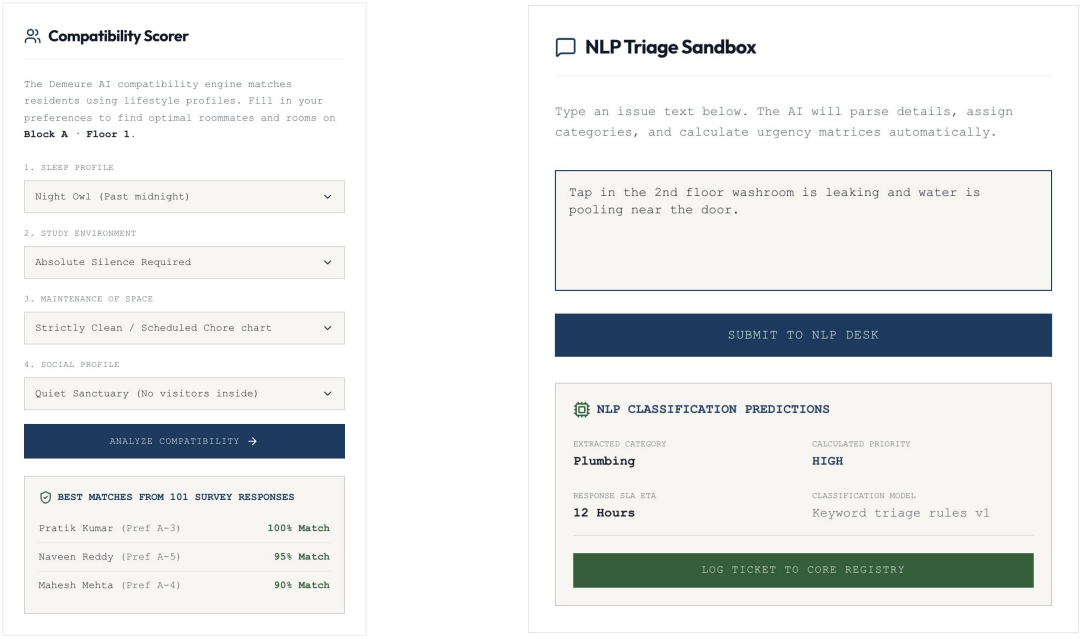


Fig. 3.8: *Survey responses and ML feature feed (101 responses)*



(a) *Compatibility scorer with ranked matches* (b) *Complaint triage prediction*

Fig. 3.9: *Decision-support engines*

3.2.6 Mess, grievances and administration

The mess manager uploads the weekly menu as an Excel sheet; it is validated and published to all students (Fig. 3.11a), and student ratings appear in the manager's feedback feed. The grievance desk accepts anonymous anti-ragging reports that staff can escalate without seeing the filer (Fig. 3.12). The power monitor lets the administrator inspect per-zone load and dispatch a new billing cycle (Fig. 3.11b), and the AWS Console Hub shows service status and a live log of every data write, e-mail and upload (Fig. 3.13).

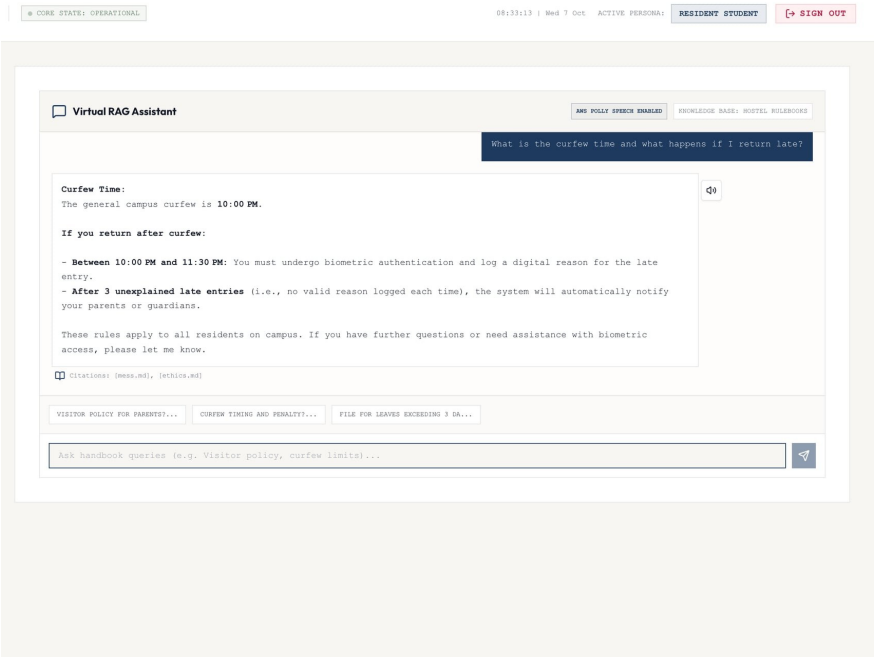
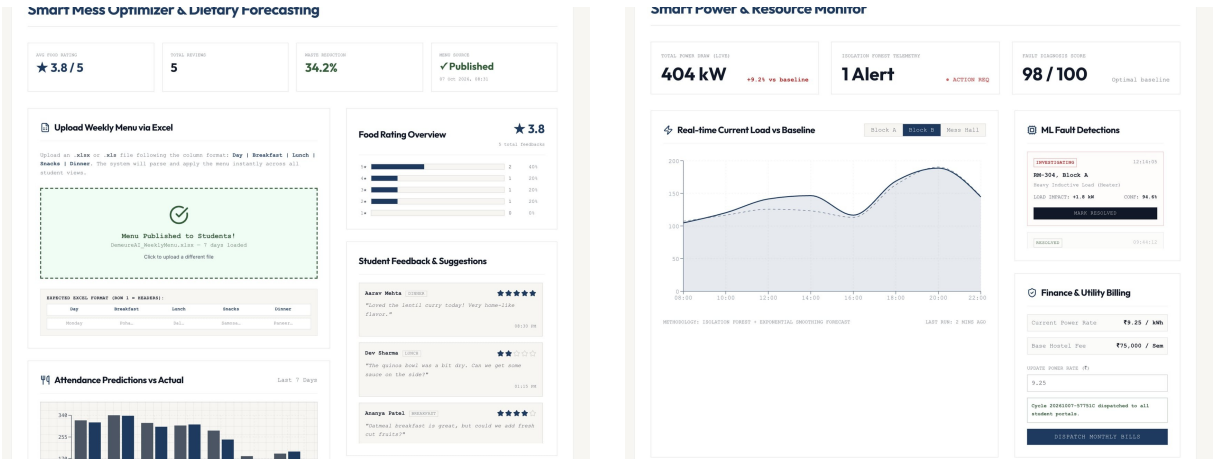


Fig. 3.10: *Retrieval-augmented handbook assistant with citations*



(a) *Weekly menu published from Excel* (b) *Power monitor and billing-cycle dispatch*

Fig. 3.11: *Mess manager and campus administrator workspaces*

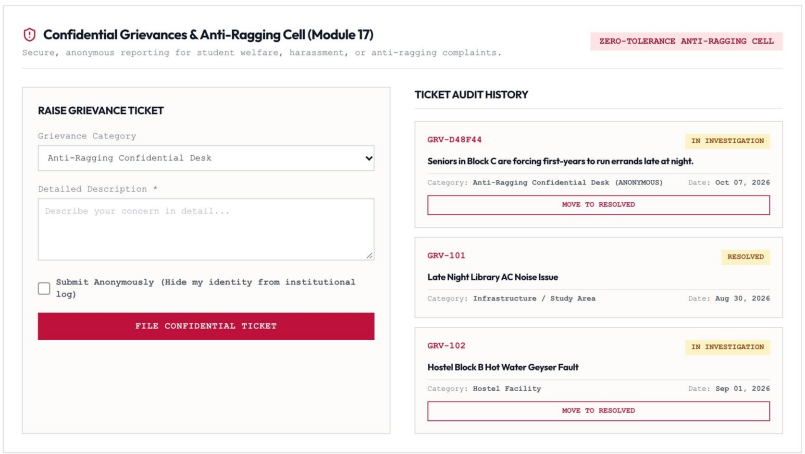


Fig. 3.12: *Anonymous grievances triaged by staff*

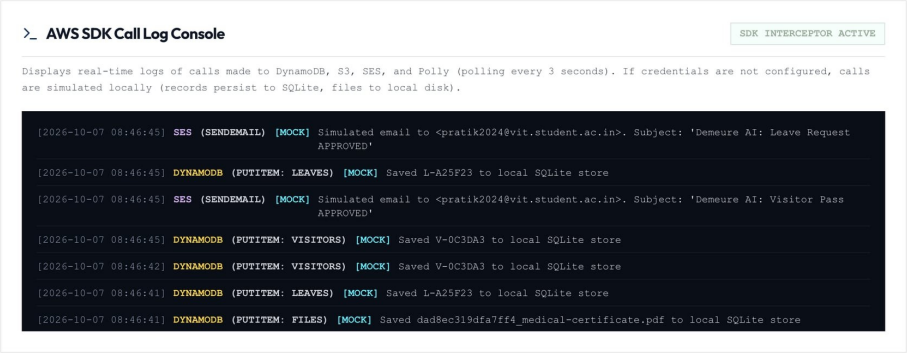


Fig. 3.13: *AWS SDK call log console*

3.2.7 Application programming interface

Table 3.3 lists representative endpoints. All request bodies are validated with Pydantic models; validation errors return HTTP 422 with a readable message. Unknown `/api` paths return a JSON 404 instead of the web page.

Table 3.3: Representative REST endpoints

Endpoint	Role	Purpose
`POST /api/auth/login, /logout; GET /api/auth/session`	Public	Sign in, sign out, restore session
`POST /api/aws/s3/upload; GET /api/files/{key}`	Signed in	Private upload; owner or staff download
`GET /api/billing/statement; POST /api/payments`	Student	Server-computed dues; UPI payment
`POST /api/billing/update`	Campus Admin	Dispatch a new billing cycle
`POST /api/leaves/submit; /api/leaves/{id}/approve|deny`	Student / Warden	Leave workflow
`POST /api/visitors; /api/visitors/{id}/approve|deny`	Student / Warden	Visitor pass workflow
`PUT /api/surveys/me; POST /api/rooms/compatibility`	Student / Warden	Survey upsert; roommate ranking
`POST /api/rooms/{room}/allocate`	Warden	Capacity-checked allocation
`POST /api/complaints/classify, /api/complaints`	Signed in	Triage preview; file complaint
`GET /api/gate/ledger; POST /api/gate/events`	Warden	Hash-chained gate log
`PUT /api/mess/menu; POST /api/mess/reviews`	Mess Manager / Student	Publish menu; rate meals
`POST /api/chat/query`	Signed in	Handbook assistant
`GET /api/health`	Public	Liveness and readiness probe

3.2.8 Deployment

A two-stage Dockerfile builds the React application with Node 22 and runs it, together with the API, in a Python 3.10 slim image as an unprivileged user. CPU-only PyTorch keeps the image small, and the embedding model is downloaded during the build so containers start without network access. A HEALTHCHECK polls `/api/health`, data is kept on a named volume, and `docker compose` starts the service in one command. In production mode the service sends Content-Security-Policy, HSTS and frame-deny headers and disables the interactive API documentation. A GitHub Actions workflow runs lint, the production build, a dependency audit and the backend tests on every push.

4. HARDWARE/SOFTWARE TOOLS USED

The project is entirely software-based. Table 4.1 lists the hardware used for development and the recommended deployment target; Table 4.2 lists the software stack with the exact versions pinned in the repository.

Table 4.1: Hardware

Item	Specification	Use
Development machine	Apple M4, 16 GB RAM, macOS 26.6	Coding, testing, model inference, screenshots
Deployment target (recommended)	1 vCPU / 2 GB RAM Linux VM, e.g. AWS EC2 t3.small, 20 GB disk	Single-container production deployment
Client devices	Any modern browser on laptop or phone	Use by all four personas

Table 4.2: Software tools and libraries

Layer	Tool / library (version)	Purpose
Frontend	React 19.2, TypeScript 6.0, Vite 8.0	Single-page application and build
	Tailwind CSS 3.4, Lucide icons, Recharts 3.8	Styling, icons, charts
	SheetJS xlsx 0.20.3	Excel import/export (menu, survey export)
Backend	Python 3.10, FastAPI 0.136, Uvicorn 0.49, Pydantic 2.13	REST API, validation, ASGI server
	SQLite (Python standard library)	Local document store
AI / ML	sentence-transformers 5.5 (all-MiniLM-L6-v2), NumPy 2.2	Embeddings and similarity search
	Groq API (openai/gpt-oss-20b); Google Gemini (optional fallback)	Grounded answer generation
Cloud	boto3 1.43: Amazon DynamoDB, S3, SES, Polly	Managed storage, files, e-mail, speech
Testing	pytest 8.4, HTTPX 0.28, Puppeteer (headless Chrome)	API tests and end-to-end scenarios
Quality	ESLint 10 with React Hooks rules, npm audit	Static analysis and dependency audit
DevOps	Docker, Docker Compose, GitHub Actions	Container image, deployment, CI
Design	Figma (wireframes)	UI wireframes for nine modules

5. RESULT ANALYSIS

The implemented system was evaluated on functional correctness, security behaviour, performance and the quality of its three intelligent components. All measurements in this chapter were taken on the development machine of Table 4.1 against the running application.

5.1 Functional verification

Ten cross-role scenarios were scripted in headless Chrome (Table 5.1). Each scenario signs in the relevant personas in separate browser contexts, performs the workflow through the user interface and checks that the other role sees the result. All ten passed from a freshly seeded database with no browser console errors other than the two deliberate negative checks (a rejected wrong PIN and a refused admin API call by a student).

Table 5.1: End-to-end scenario results

#	Scenario	Roles	Result
1	Leave request with PDF proof; warden opens the private file and approves	Student, Warden	Pass
2	Visitor pass request; approval issues a gate pass code	Student, Warden	Pass
3	Billing-cycle dispatch; student pays server-computed dues; receipt stored	Admin, Student	Pass
4	Survey submission joins 100 seeded responses; ranking; capacity-checked allocation	Student, Warden	Pass
5	Complaint triage and status workflow (Open, Dispatched, Resolved)	Student, Warden	Pass
6	Handbook question answered with citations	Student	Pass
7	Weekly menu published from Excel; student rating reaches the manager	Mess Manager, Student	Pass
8	Anonymous grievance escalated without exposing the filer	Student, Admin	Pass
9	Facility booking approval and maintenance work-order progression	Student, Admin	Pass
10	Biometric gate events appended to the verified hash chain	Warden	Pass

5.2 Automated tests

The backend test suite contains 32 tests that run against an isolated temporary database (Table 5.2). They cover the security model as well as the business rules; for example, a session cookie whose payload is edited to claim the administrator role is rejected, and production configuration with demo PINs refuses to start.

Table 5.2: Backend test coverage by area (32 tests, all passing)

Area	What is verified
Authentication and sessions	Login success and failure, logout, forged-role token rejected, session probe
Authorisation	Unauthenticated access returns 401; cross-role and cross-student access returns 403
Leaves and files	Attachment ownership, date validation, private file access, one-way approval (409)
Uploads	Rejects wrong type, spoofed content and files over 5 MB (413)
Billing and payment	Server-side totals, duplicate payment refused, new cycle reopens dues, rate validation
Surveys and rooms	Upsert, 100-student seed, ranking order, single-gender blocks, capacity, unknown rooms
Campus workflows	Anonymous grievances, complaint triage and status, bookings, tickets, mess menu and reviews
Ledger and configuration	Hash-chain verification detects tampering; production configuration guard

5.3 API performance

Seven representative endpoints were called sequentially over HTTP on localhost after a warm-up request (Table 5.3, Fig. 5.1). Every data endpoint completed within 3.1 ms at the 95th percentile, including listing all 101 survey responses and ranking roommates across them. Login is intentionally slower (about 27 ms) because PBKDF2 with 120,000 iterations makes password guessing expensive; this cost is paid once per session.

Table 5.3: Endpoint latency (milliseconds)

Endpoint	Requests	Median	95th percentile
Login (PBKDF2 verify)	50	27.09	30.15
Billing statement	200	1.85	2.16
Student leaves	200	1.71	2.36
Complaint triage	200	2.01	2.46
Survey list (101 rows)	200	2.52	2.81
Roommate ranking	200	2.79	3.00
Gate ledger + verify	200	1.84	2.07

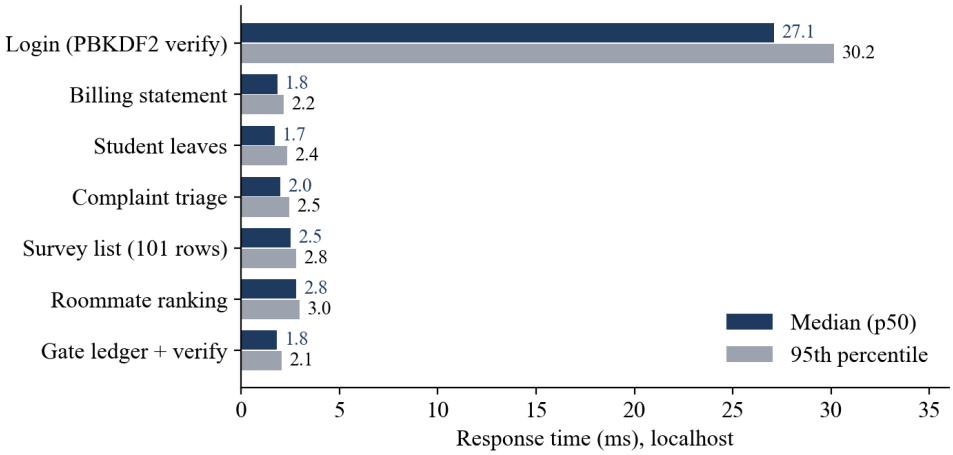


Fig. 5.1: *Median and 95th-percentile response times*

5.4 Handbook assistant

Eighteen questions were written, one for each policy section in the six rulebooks, each with a known source file and an expected fact (for example, the curfew time or the mess wastage fine). Retrieval returned the correct rulebook as the top result for **18 of 18** questions, even though each rulebook is padded with about one million words of generic annexure text. The generated answer contained the expected fact for **17 of 18** questions. In the remaining case ("Who has to provide detergent for laundry?") the correct file was retrieved but not the specific section, and the model correctly said that the provided excerpts did not answer the question rather than guessing. Median retrieval time was 24 ms and the median end-to-end answer time, including the Groq API call, was 0.61 s (maximum 1.13 s), as summarised in Table 5.4.

Table 5.4: Handbook assistant evaluation (18 questions)

Metric	Result
Correct rulebook retrieved at rank 1	18/18 (100%)
Correct rulebook within top 2	18/18 (100%)
Answer contains the expected fact	17/18 (94.4%)
Median retrieval time	24 ms
Median / maximum answer time (with LLM)	0.61 s / 1.13 s
Index size and load time	30 chunks; 31 s cold start (background thread)

5.5 Roommate compatibility on the 100-student dataset

For each of the 100 seeded students, the other 99 were ranked with the student's own answers as the query. The 100 students have 56 distinct lifestyle profiles. Across all 9,900 ordered pairs the mean score is 53.9, but the best match per student averages 96.7 with a minimum of 85, so every student has at least one highly compatible candidate (Fig. 5.2).

The analysis also exposed a real defect. Before the block constraint was added, the best match was of the same gender for only 45 of 100 students, because the score ignores gender while Blocks A and B are single-gender. After restricting candidates to the target block, Block A matches were same-gender for 44 of 44 students and Block B for 37 of 37. Block C is mixed by design. The mean best-match score changed only from 97.0 to 96.7.

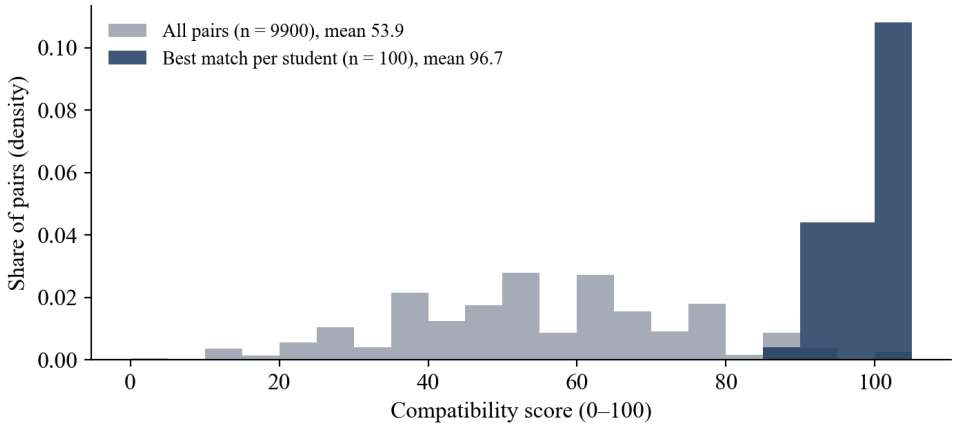


Fig. 5.2: *Distribution of compatibility scores: all pairs versus each student's best match*

5.6 Complaint triage

A labelled sample of 30 realistic complaints (six per category: Electrical, Plumbing, IT/WiFi, Structural and General) was used. The first version of the rules, which matched keywords anywhere inside words, classified 28 of 30 correctly (93.3%). Both errors were substring false positives: "WiFi keeps disconnecting in Block A" was labelled Structural because "Block" contains "lock", and "Need an extra blanket for winter" was labelled IT / WiFi because "blanket" contains "lan". In addition, "switch board gives a shock" was not escalated as critical. Matching was changed to word-start matching and "shock" was added as a hazard term. The revised rules classify 30 of 30 correctly and flag both hazards as critical (Fig. 5.3, left). Because the same sample guided the fix, this result is optimistic, and a larger held-out set of real campus complaints is needed for an unbiased estimate (Section 6.2).

5.7 Engineering quality

Table 5.5 compares the hackathon prototype with this release. Route-level code splitting cut the JavaScript downloaded on first visit by 81% (Fig. 5.3, right): modules such as the Excel parser and the charting library load only on the screens that use them.

Table 5.5: Prototype versus production release

Measure	Prototype	This release
Server-side authentication and authorisation	None (PINs checked in the browser)	Hashed credentials, signed sessions, role checks on every route
Data persistence	In memory, lost on restart	SQLite or DynamoDB, durable
ESLint errors	65	0
High-severity production dependency advisories	1 (xlsx 0.18.5)	0
Automated tests	0	32 (plus 10 end-to-end scenarios)
First-load JavaScript (raw / gzip)	1,316 kB / 386 kB	254 kB / 76 kB
Deployment artefacts	None	Dockerfile, Compose file, CI workflow, health check

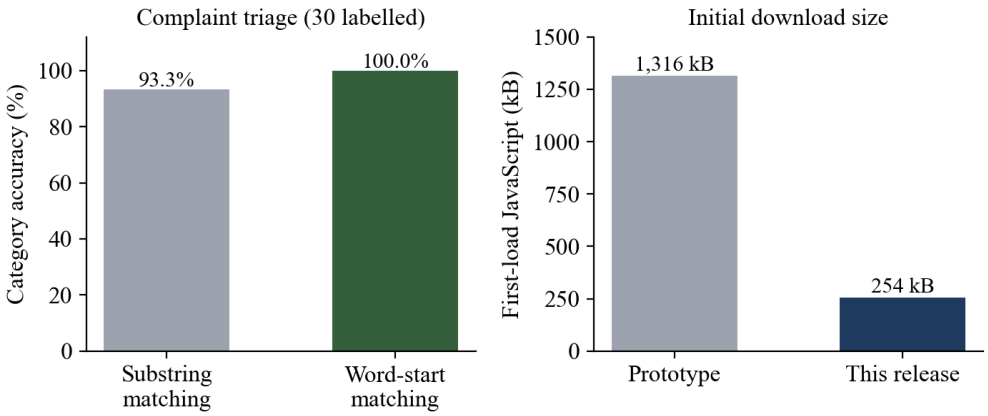


Fig. 5.3: *Triage accuracy before and after the matching fix (left); first-load download size (right)*

5.8 Discussion

The results show that a modest stack with no GPU and a single small server is enough for the platform's workloads: data endpoints respond in milliseconds and the assistant answers in well under two seconds. The evaluation also proved valuable as a debugging tool: measuring the matching engine on the 100-student dataset revealed the cross-gender recommendation problem, and the labelled triage sample revealed substring false positives. Both were fixed and are now covered by automated tests. One inconsistency remains to be resolved with the administration: the room rulebook promises a four-hour response for critical electrical or water faults, while the triage engine targets two hours, which is stricter than the published policy.

6. CONCLUSION AND FUTURE WORK

6.1 Summary

Demeure AI turns a hackathon prototype into a deployable campus operating system. It unifies leave and visitor approvals, room allocation, complaints, mess planning, grievances, facilities, billing and payment for four personas on one persisted record. Security is enforced on the server through hashed credentials, signed sessions, role and ownership checks, validated input and private file storage. The hybrid design runs on a single machine or on AWS DynamoDB, S3, SES and Polly without code changes. The intelligent features measured well: the handbook assistant retrieved the right rulebook for every test question and answered 17 of 18 correctly; roommate matching gives every student a best match scoring at least 85 while respecting single-gender blocks; and complaint triage classified all 30 labelled complaints after the word-matching fix. Ten end-to-end scenarios and 32 automated tests pass, and core API calls complete within 3 ms.

6.2 Limitations and Constraints

- **Payments are simulated.** The UPI flow records a payment and receipt but is not connected to a payment gateway.
- **Simulated data sources.** Power telemetry, biometric gate scans and academic records (timetable, grades, library) are sample data; no meters, scanners or student information system are connected.
- **Small evaluation sets.** Triage was evaluated on 30 hand-written complaints that also guided the fix, and the assistant on 18 questions; larger held-out sets of real campus data are needed.
- **Rule-based triage.** Keyword rules are transparent but cannot understand paraphrases that use none of the keywords; the 2-hour critical target is stricter than the rulebook's 4-hour commitment.
- **Single-node local mode.** The SQLite store suits one server; multi-server deployments need the DynamoDB mode.
- **Deployment validation.** The container image and live AWS mode were not exercised on a cloud account during this evaluation, which ran in local mode.
- **Third-party AI.** Answer quality and latency depend on the external LLM provider, and the assistant returns raw policy text if the provider is unavailable.

6.3 Improvement / Future Work

- Integrate a UPI payment gateway such as Razorpay or PayU with signed webhooks for reconciliation.
- Connect real data sources: smart energy meters for per-room billing, biometric gate hardware posting to the ledger API, and the university's student information system for academics.
- Replace keyword triage with a fine-tuned or embedding-based classifier trained on real tickets, keeping the rules as an explainable fallback.
- Add campus single sign-on (Google Workspace or Microsoft Entra ID) and multi-factor authentication for staff.
- Section-level retrieval with re-ranking to fix misses such as the laundry detergent question.
- Constraint-aware stable roommate matching [19] for bulk allocation at the start of each semester.
- Push notifications and a mobile app; Hindi and regional-language interfaces for wider accessibility.

7. SOCIAL AND ENVIRONMENTAL IMPACT

Student safety and well-being. Anonymous anti-ragging reporting implements the confidential channel expected by the UGC regulations [22] while keeping each case accountable through tracked status. Welfare alerts help wardens notice residents who may need support, and hazardous complaints such as sparking sockets are escalated automatically.

Transparency and trust. Every approval is a recorded state change, students can track their own requests, and the hash-chained gate ledger makes tampering with security records detectable. This supports accountable institutions (SDG 16.6).

Paperless operations. Leave, visitor, booking and complaint forms that were previously printed and signed become digital records, which reduces paper use and the time students spend queuing at offices.

Energy and food. Per-zone power monitoring and usage-based billing make consumption visible to administrators and students (SDG 7.3). Published menus, meal ratings and wastage fines give mess managers the feedback needed to cut food waste (SDG 12.3) [24].

Low-cost, low-footprint deployment. The whole platform runs on one small CPU-only server, and the AI component uses a compact embedding model, so the system can be adopted by institutions with limited budgets.

Privacy and ethics. The system collects only operational data. Uploaded documents are private to the student and authorised staff, anonymous grievances never expose the filer to staff, and the welfare module states that no personal devices or audio are monitored. Wardens remain responsible for every decision; the compatibility and triage outputs are recommendations, not automated decisions.

Accessibility. Custom dropdowns and rating controls expose ARIA roles and labels for screen readers, and assistant answers can be read aloud.

8. WORK PLAN

8.1 Timeline

The project ran from June to October 2026 in the phases shown in Table 8.1.

Table 8.1: Project timeline (2026)

Period	Phase	Key outputs
Early June	Problem study and design	Requirements, Figma wireframes for nine modules, 100-student survey dataset
13–14 June	WebForge hackathon prototype	React modules, FastAPI back end, RAG assistant over hostel rulebooks
July – August	AWS integration	DynamoDB, S3, SES and Polly adapters; AWS Console Hub; hybrid mock mode
September	Feature expansion	Academic, facilities and finance hubs; documentation
October (weeks 1–2)	Production hardening	Server-side auth, persistence for all modules, validation, missing workflows
October (weeks 2–3)	Testing and evaluation	32 tests, 10 end-to-end scenarios, latency and accuracy measurements
October (week 3)	Deployment and reporting	Docker image, CI pipeline, presentation and this report

8.2 Individual Contribution

Table 8.2 lists each member's contribution.

Table 8.2: Individual contribution

Member	Register No.	Contribution
Pratik Kumar	24BML0027	Project lead; system architecture; FastAPI back end, authentication and access control; AWS integration; RAG assistant
Thakur Chand Choudhary	24BML0012	Front-end modules (student services, security console, complaint desk, mess planner); UI testing and end-to-end scenarios
Nikunj Goydani	24BEC0338	Survey dataset and roommate compatibility; complaint triage evaluation; deployment (Docker, CI); documentation and report

9. COST ANALYSIS

Development cost. All software used is open source or free for development (Table 4.2), and development used the team's existing laptops, so the development cost was nil apart from internet access. In local mode the platform needs no paid services.

Operating cost. Table 9.1 estimates the monthly cost of running Demeure AI on AWS for a hostel of about 1,000 students. The workload assumptions are 2 million reads and 0.3 million writes per month, 10 GB of stored documents, 20,000 notification e-mails, 2 million characters of speech synthesis and 30,000 assistant questions. Prices are indicative on-demand list prices for the Mumbai (ap-south-1) region at the time of writing, converted at ₹ 85 per US dollar; they should be confirmed with the AWS Pricing Calculator before procurement.

Table 9.1: Indicative monthly operating cost on AWS (1,000 students)

Item	Basis	USD / month	INR / month
Compute: EC2 t3.small + 20 GB gp3 disk	730 hours, on demand	18.20	1,547
Amazon DynamoDB (on-demand)	2 M reads, 0.3 M writes, < 1 GB stored	1.30	111
Amazon S3 (private documents)	10 GB Standard storage + requests	0.30	26
Amazon SES (e-mail)	20,000 e-mails at $0.10 per 1,000	2.00	170
Amazon Polly (standard voices)	2 M characters at $4 per million	8.00	680
LLM API for the assistant	30,000 questions, ~1,400 tokens each	≈ 7.00	≈ 595
Domain and DNS (Route 53)	Hosted zone; TLS certificate free (ACM)	0.60	51
Total		**≈ 37.40**	**≈ 3,180**

This is about ₹ 3.2 per student per month, or roughly ₹ 38 per student per year. Running in local mode on an existing campus server removes every line except the LLM API, and the assistant can fall back to returning retrieved policy text at no cost. AWS Free Tier allowances further reduce the cost for the first year.

10. REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[3] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 5776–5788.

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, "Attention is all you need," in *Proc. Advances in Neural Information Processing Systems (NIPS)*, vol. 30, 2017, pp. 5998–6008.

[5] F. T. Liu, K. M. Ting and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.

[6] S. Haber and W. S. Stornetta, "How to time-stamp a digital document," *Journal of Cryptology*, vol. 3, no. 2, pp. 99–111, 1991.

[7] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: `https://bitcoin.org/bitcoin.pdf`

[8] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, Aug. 2015.

[9] H. Krawczyk, M. Bellare and R. Canetti, "HMAC: Keyed-hashing for message authentication," IETF RFC 2104, Feb. 1997.

[10] K. Moriarty, B. Kaliski and A. Rusch, "PKCS #5: Password-based cryptography specification version 2.1," IETF RFC 8018, Jan. 2017.

[11] A. Barth, "HTTP state management mechanism," IETF RFC 6265, Apr. 2011.

[12] P. A. Grassi, J. L. Fenton, E. M. Newton *et al.*, "Digital identity guidelines: Authentication and lifecycle management," NIST Special Publication 800-63B, Jun. 2017.

[13] OWASP Foundation, "OWASP Top 10:2021," 2021. [Online]. Available: `https://owasp.org/Top10/`

[14] M. West, "Content Security Policy Level 3," W3C Working Draft, World Wide Web Consortium. [Online]. Available: `https://www.w3.org/TR/CSP3/`

[15] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall and W. Vogels, "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM Symp. Operating Systems Principles (SOSP)*, 2007,

pp. 205–220.

[16] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, Univ. of California, Irvine, 2000.

[17] Amazon Web Services, “AWS Well-Architected Framework,” AWS Whitepaper, 2023. [Online]. Available: https://docs.aws.amazon.com/wellarchitected/latest/framework/

[18] D. Gale and L. S. Shapley, “College admissions and the stability of marriage,” *The American Mathematical Monthly*, vol. 69, no. 1, pp. 9–15, 1962.

[19] R. W. Irving, “An efficient algorithm for the ‘stable roommates’ problem,” *Journal of Algorithms*, vol. 6, no. 4, pp. 577–595, 1985.

[20] Z. Min-Allah and S. Alrashed, “Smart campus—A sketch,” *Sustainable Cities and Society*, vol. 59, Art. no. 102231, 2020.

[21] A. Zanella, N. Bui, A. Castellani, L. Vangelista and M. Zorzi, “Internet of Things for smart cities,” *IEEE Internet of Things Journal*, vol. 1, no. 1, pp. 22–32, Feb. 2014.

[22] University Grants Commission, “UGC Regulations on Curbing the Menace of Ragging in Higher Educational Institutions, 2009,” New Delhi, India, 2009.

[23] United Nations General Assembly, “Transforming our world: The 2030 Agenda for Sustainable Development,” Resolution A/RES/70/1, 2015.

[24] United Nations Environment Programme, “Food Waste Index Report 2021,” Nairobi, Kenya, 2021.

[25] OpenAI, “gpt-oss-120b & gpt-oss-20b model card,” Aug. 2025. [Online]. Available: https://openai.com/index/gpt-oss-model-card/

[26] Amazon Web Services, “Amazon DynamoDB Developer Guide,” “Amazon S3 User Guide,” “Amazon SES Developer Guide” and “Amazon Polly Developer Guide,” AWS Documentation. [Online]. Available: https://docs.aws.amazon.com/

11. APPENDIX-1: SOURCE CODE

The complete source code (React front end, FastAPI back end, tests, Dockerfile and CI workflow) is in the project repository. This appendix reproduces the core algorithms of Section 3.1.4.

A.1 Business rules: triage, compatibility and ledger (`backend/services/domain.py`)

```
# --- Complaint triage ------------------------------------------------------
# The category with the most keyword hits wins; ties go to the earlier (more
# safety-critical) category, so "water on the socket" is Electrical, not Structural.
_CATEGORY_RULES = [
    ("Electrical", "HIGH", "8 Hours", ("electric", "power", "fuse", "shock", "wire", "light
    ↪ ", "fan",
                                       "switch", "socket", "spark")),
    ("Plumbing", "HIGH", "12 Hours", ("water", "leak", "pipe", "clog", "drain", "tap", "
    ↪ flush", "drip")),
    ("Structural", "MEDIUM", "24 Hours", ("ceiling", "crack", "door", "lock", "window", "
    ↪ wall", "bed")),
    ("IT / WiFi", "MEDIUM", "24 Hours", ("wifi", "wi-fi", "router", "internet", "connection
    ↪ ", "lan", "network")),
]
_CRITICAL_TERMS = ("socket", "dripping", "fuse board", "fire", "smoke", "spark", "shock", "
    ↪ emergency", "flood", "gas")


def _has(term: str, text: str) -> bool:
    # Terms match at the start of a word ("leak" -> "leaking"), never inside one
    # ("lock" must not match "Block", "lan" must not match "blanket").
    return re.search(r"\b" + re.escape(term), text) is not None


def triage_complaint(text: str) -> dict:
    lower = text.lower()
    result = {"category": "General", "priority": "LOW", "eta": "48 Hours"}
    best_hits = 0
    for category, priority, eta, terms in _CATEGORY_RULES:
        hits = sum(_has(term, lower) for term in terms)
        if hits > best_hits:
            best_hits = hits
            result = {"category": category, "priority": priority, "eta": eta}
    if any(_has(term, lower) for term in _CRITICAL_TERMS):
        result["priority"] = "CRITICAL (IMMEDIATE)"
        result["eta"] = "2 Hours"
    return result
```

```
# --- Roommate compatibility ------------------------------------------------
# Both the allocation form (short codes) and the survey (labels) map onto the
# same 0..2 ordinal scale so the two vocabularies are comparable.
_SCALES = {
    "sleep": {"early": 0, "early riser": 0, "flexible": 1, "night": 2, "night owl": 2},
    "study": {"silent": 0, "absolute silence": 0, "external": 1, "library preferred": 1,
              "ambient": 2, "ambient / music ok": 2},
    "clean": {"strict": 0, "strict / scheduled": 0, "moderate": 1, "moderate / weekends":
    ↪ 1,
              "relaxed": 2, "relaxed / minimal": 2},
    "social": {"quiet": 0, "quiet sanctuary": 0, "moderate": 1, "moderate / occasional": 1,
               "active": 2, "active hub": 2},
}
_WEIGHTS = {"sleep": 0.30, "study": 0.25, "clean": 0.25, "social": 0.20}
_SURVEY_FIELDS = {"sleep": "sleepHabit", "study": "studyEnv", "clean": "cleanliness", "
    ↪ social": "socialPref"}


def _ordinal(dimension: str, value: str):
    return _SCALES[dimension].get((value or "").strip().lower())


def compatibility_score(prefs: dict, survey: dict) -> int:
    """Weighted similarity (0–100) between requested preferences and one survey entry."""
    total, weight_seen = 0.0, 0.0
    for dim, weight in _WEIGHTS.items():
        a = _ordinal(dim, prefs.get(dim, ""))
        b = _ordinal(dim, survey.get(_SURVEY_FIELDS[dim], ""))
        if a is None or b is None:
            continue
        total += weight * (1 - abs(a - b) / 2)
        weight_seen += weight
    return round(100 * total / weight_seen) if weight_seen else 0


# Block A houses men and Block B women; Block C is mixed / PG scholars.
GENDERED_BLOCKS = {"A", "B"}


def rank_roommates(prefs: dict, surveys: list[dict], block: str = "", limit: int = 3) ->
    ↪ list[dict]:
    ranked = []
    if block in GENDERED_BLOCKS:
        # Only residents who chose this single-gender block are eligible roommates there.
        surveys = [s for s in surveys if s.get("preferredBlock") == block]
    for s in surveys:
```

```
        score = compatibility_score(prefs, s)
        if block and s.get("preferredBlock") == block:
            score = min(100, score + 5)  # small bonus for wanting the same block
        ranked.append({
            "name": s.get("fullName", "Unknown"),
            "score": score,
            "room": s.get("allocatedRoom") or f"Pref {s.get('preferredBlock', '?')}-{s.get
    ↪ ('preferredFloor', '?')}",
            "studentId": s.get("studentId", ""),
        })
    ranked.sort(key=lambda r: r["score"], reverse=True)
    return ranked[:limit]


# --- Gate ledger ------------------------------------------------------------
GENESIS_HASH = "0" * 64


def ledger_hash(prev_hash: str, entry: dict) -> str:
    """SHA-256 over the previous hash and the entry's canonical fields (a hash chain)."""
    fields = {k: entry[k] for k in ("time", "name", "type", "roomOrPurpose", "status")}
    payload = prev_hash + json.dumps(fields, sort_keys=True, separators=(",", ":"))
    return hashlib.sha256(payload.encode("utf-8")).hexdigest()


def verify_ledger(entries_oldest_first: list[dict]) -> bool:
    prev = GENESIS_HASH
    for entry in entries_oldest_first:
        if entry.get("prevHash") != prev or ledger_hash(prev, entry) != entry.get("hash"):
            return False
        prev = entry["hash"]
    return True
```

A.2 Signed session tokens (`backend/services/auth.py`, excerpt)

```
    # --- tokens -----------------------------------------------------------
    def issue_token(self, user: SessionUser) -> str:
        payload = {**user.public(), "exp": int(time.time()) + self._ttl}
        body = _b64(json.dumps(payload, separators=(",", ":")).encode("utf-8"))
        sig = _b64(hmac.new(self._key, body.encode("ascii"), hashlib.sha256).digest())
        return f"{body}.{sig}"

    def verify_token(self, token: str) -> Optional[SessionUser]:
        try:
            body, sig = token.split(".", 1)
```

```
            expected = _b64(hmac.new(self._key, body.encode("ascii"), hashlib.sha256).
    ↪ digest())
            if not hmac.compare_digest(sig, expected):
                return None
            payload = json.loads(_unb64(body))
            if int(payload.pop("exp", 0)) < time.time():
                return None
            if payload.get("role") not in ROLES:
                return None
            return SessionUser(**payload)
        except (ValueError, TypeError, json.JSONDecodeError):
            return None
```

A.3 Role guard dependency (`backend/deps.py`, excerpt)

```
def current_user(request: Request, services: Services = Depends(get_services)) ->
    ↪ SessionUser:
    token = request.cookies.get(SESSION_COOKIE)
    user = services.auth.verify_token(token) if token else None
    if not user:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Not signed in or session expired
    ↪ ")
    return user


def require_roles(*roles: str) -> Callable[..., SessionUser]:
    def dependency(user: SessionUser = Depends(current_user)) -> SessionUser:
        if user.role not in roles:
            raise HTTPException(status.HTTP_403_FORBIDDEN, "Your role cannot perform this
    ↪ action")
        return user
    return dependency
```